

Lab 6: Inheritance

Theory:

- Define inheritance, base and derived class.
- Explain different types of inheritance in brief.

Lab:

Question: 1

WAP in C++ to show the concept of overriding in inheritance.

```
#include<iostream>
using namespace std;
class Base{
    protected:
        int n;
    public:
        void input(){
            cout<<"Enter the value to the n of a base class:"<<endl;
            cin>>n;
        }
        void display(){
            cout<<"The value to the base of n="<<n<<endl;
        }
};

class Derived:public Base{
    protected:
        int n;
    public:
        void input( ){
            cout<<"Enter the value to the n of a derived class:"<<endl;
            cin>>n;
        }
        void display( )
        {
            cout<<"The value to the n of a derived class is:"<<n<<endl;
            cout<<"The value of both the n is :"<<n+n<<endl;
        }
}
```

```
};

int main( )
{
    Derived d;
    d.input(); //with the intention of calling Base input()
    d.display( ); //with the intention of calling Base display()
    d.input( ); //with the intention of calling Derived input()
    d.display( ); //with the intention of calling Derived display()
    return 0;
}
```

Output:

Question: 2

WAP in C++ to show the concept of single inheritance.

```
#include<iostream>
using namespace std;
class employee{
protected:
    char name[20];
    float salary;
public:
    void input( )
    {
        cout<<"Enter the name of an employee:"<<endl;
        cin>>name;
        cout<<"Enter the salary of an employee:"<<endl;
        cin>>salary;
    }
    void display( ){
        cout<<"Name of an employee="<<name<<endl;
        cout<<"Salary of an employee="<<salary<<endl;
    }
};
class manager:public employee
{
protected:
```

```

        char title[20];
public:
void input( )
{
    employee::input( );
    cout<<"Enter the title of an manager:"<<endl;
    cin>>title;
}
void display( ){
    employee::display( );
    cout<<"The title of a manager="<<title<<endl;
}
};
int main( )
{
    manager m1,m2;
    m1.input( );
    m2.input( );
    m1.display( );
    m2.display( );
    return 0;
}

```

Output:

Question: 3

WAP in C++ to show the concept of multiple inheritance.

```

#include<iostream>
using namespace std;
class Base1
{
protected:
int x;
public:
void get_x( )
{
    cout<<"Enter the value to x of Base1:"<<endl;
    cin>>x;
}
void display_x( )

```

```

{
    cout<<"The value to x of Base1="<<x<<endl;
}
};

class Base2
{
    protected:
        int y;
    public:
        void get_y( )
        {
            cout<<"Enter the value to y of a Base2:"<<endl;
            cin>>y;
        }
        void display_y( )
        {
            cout<<"The value to the y of a Base2="<<y<<endl;
        }
};

class derived: public Base1,public Base2
{
    protected:
        int z;
    public:
        void get_z( )
        {
            cout<<"Enter the value to z of a derived class:"<<endl;
            cin>>z;
        }
        void display_z( )
        {
            cout<<"The value to the z of a derived class="<<z<<endl;
        }
        void display( )
        {
            cout<<"The sum is:"<<x+y+z<<endl;
        }
};

int main( )
{
    derived d;
    d.get_x( );
}

```

```

        d.get_y( );
        d.get_z( );
        d.display_x( );
        d.display_y( );
        d.display_z( );
        d.display( );
        return 0;
    }

```

Output:

Question: 4

WAP in C++ to show the concept of multilevel inheritance.

```

#include<iostream>
using namespace std;
class student
{
protected:
    char name[20];
    int roll;
public:
    void input( )
    {
        cout<<"Enter the name of an student:"<<endl;
        cin>>name;
        cout<<"Enter the roll number of an student:"<<endl;
        cin>>roll;
    }
    void display( )
    {
        cout<<"The name of an student="<<name<<endl;
        cout<<"The roll number of an student="<<roll<<endl;
    }
};
class test:public student
{
protected:

```

```

        float sub1,sub2;
public:
void input( )
{
    student::input( );
    cout<<"Enter the marks in subject1 and subject2:"<<endl;
    cin>>sub1>>sub2;
}
void display( )
{
    student::display( );
    cout<<"The marks of subject1="<<sub1<<endl;
    cout<<"The marks of subject2="<<sub2<<endl;
}
};

class result: public test
{
protected:
    float total; public:
void input( )
{
    test::input( );
    total=sub1+sub2;
}
void display( )
{
    test::display( );
    cout<<"Total="<<total<<endl;
}
};

int main( )
{
    result r;
    r.input( );

```

```

        r.display( );
        return 0;
    }

```

Output:

Question: 5

WAP in C++ to show the concept of hierarchical inheritance.

```

#include<iostream>
using namespace std;
class employee{
protected:
    char name[20];
    float salary;
public:
    void input( ){
        cout<<"Enter the name of an employee:"<<endl;
        cin>>name;
        cout<<"Enter the salary of an employee:"<<endl;
        cin>>salary; }
    void display( ){
        cout<<"Name of an employee="<<name<<endl;
        cout<<"Salary of an employee="<<salary<<endl;
    }
};
class manager: public employee
{
protected:
    char title[20];
public:
    void input( )
    {
        employee::input( );
        cout<<"Enter the title of a manager:"<<endl;
        cin>>title;
    }
}

```

```

}
void display( )
{
    employee::display( );
    cout<<"The title of a manager="<<title<<endl;
}
};
class teacher:public employee
{
protected:
    char faculty[20];
public:
void input( )
{
    employee::input( );
    cout<<"Enter the faculty of the teacher:"<<endl; cin>>faculty;
}
void display( )
{
    employee::display( );
    cout<<"The faculty of the teacher="<<faculty;
}
};
int main( )
{
    cout<<"Manager:"<<endl;
    manager m;
    m.input( );
    cout<<"Teacher:"<<endl;
    teacher t;
    t.input( );
    cout<<"Manager details:"<<endl;
    m.display( );
    cout<<"Teacher details:"<<endl;
}

```



```

        t.display( );
    return 0;
}

```

Output:

Question: 6

WAP in C++ to show the concept of hybrid inheritance.

```

#include<iostream>
using namespace std;
class student
{
protected:
    char name[20];
    int roll;
public:
void input( )
{
    cout<<"Enter the name of the student:"<<endl;
    cin>>name;
    cout<<"Enter the roll number of the student:"<<endl;
    cin>>roll;
}
void display( )
{
    cout<<"Name of the student="<<name<<endl;
    cout<<"Roll number of the student="<<roll<<endl;
}
};
class test: public student
{
protected:
    float sub1,sub2;
public:
void input( )

```

```

{
    student::input( );
    cout<<"Enter the marks in subject1 and subject2:"<<endl;
    cin>>sub1>>sub2;
}
void display( )
{
    student::display( );
    cout<<"The marks of subject1="<<sub1<<endl;
    cout<<"The marks of subject2="<<sub2<<endl;
}
};
class sports
{
protected:
    float sm;
public:
void input( )
{
    cout<<"Enter the marks of sports:"<<endl;
    cin>>sm;
}
void display( )
{
    cout<<"The marks of the sports="<<sm<<endl;
}
};
class result: public test, public sports
{
protected:
    float total;
public:
void input( )
{

```

```

        test::input( );
        sports::input( );
        total=sub1+sub2+sm;
    }
void display( )
{
    test::display( );
    sports::display( );
    cout<<"Total marks="<<total<<endl;
}
};
int main( )
{
    result r;
    r.input( );
    r.display( );
    return 0;
}

```

Output:

Question: 7

WAP in C++ to show the concept of constructor in inheritance.

```

#include<iostream>
using namespace std;
class alpha
{
protected:
    int x;
public:
    alpha(int i)
    {
        x=i;
        cout<<"alpha initialized"<<endl;
    }
}

```

```

void display_x( )
{
    cout<<"x="<<x<<endl;
}
};

class beta
{
protected:
    int y;
public:
    beta( int j)
    {
        y=j;
        cout<<"beta initialized"<<endl;
    }
    void display_y( )
    {
        cout<<"y="<<y<<endl;
    }
};

class gamma: public alpha, public beta
{
protected:
    int m,n;
public:
    gamma(int a,int b,int c,int d):alpha(a),beta(b)
    {
        m=c;
        n=d;
        cout<<"gamma initialized"<<endl;
    }
};

```

```

void display_mn( )
{
    cout<<"m="<<m<<endl;
    cout<<"n="<<n<<endl;
}
};

int main( )
{
gamma g(11,12,13,14);
g.display_x( );
g.display_y( );
g.display_mn( );
return 0;
}

```

Output:

Question: 8

WAP in C++ to show the concept of object slicing.

```

#include<iostream>
using namespace std;
class Base {
public:
    int a;
    void show()
    {
        cout << "Base: " << a << endl;
    }
};

class Derived : public Base {
public:
    int b;
    void show()

```

```
    {  
        cout << "Derived: " << a << ", " << b << endl;  
    }  
};  
  
int main() {  
    Derived d;  
    d.a = 1;  
    d.b = 2;  
    Base b = d; // Slicing: b has only 'a'  
    b.show();   // Prints: Base: 1  
}
```

Note:

- A **Derived object** is assigned to a **Base object**.
- This is **object slicing**:
 - Only the Base portion (a) is copied into b.
 - The Derived-specific part (b) is **sliced off** (lost).

Output:

Conclusion:

- What did you learn from the lab?